

Ecualizador digital

Cherro Cristian, Fuster Lucio

Departamento de electrónica, Universidad Tecnológica Nacional FRA

Buenos Aires, Argentina

luciofuster@gmail.com

cristiancherro@hotmail.com.ar

Abstract— Este proyecto presenta el diseño e implementación de un ecualizador digital que procesa señales de audio mediante un microcontrolador ESP32 usando el sistema operativo en tiempo real FreeRTOS. El sistema divide la señal en tres bandas con ajuste de ganancia en cada una. Se integran módulos de adquisición analógica, procesamiento digital con algoritmos DSP y conversor para salida analógica.

ÍNDICE

I. INTRODUCCIÓN.....	1
II. OBJETIVOS.....	1
III. DIAGRAMA EN BLOQUES.....	1
IV. HARDWARE.....	1
A. ADC I ² S.....	1
B. DAC I ² S.....	2
C. LCD 20x04.....	2
D. Encoder rotativo.....	2
E. UART PC.....	2
F. ESP 32.....	3
VI. SOFTWARE.....	3
VII. CONCLUSIONES.....	4
REFERENCIAS.....	4
ANEXOS.....	5

I. INTRODUCCIÓN

El procesamiento digital de señales (DSP) aplicado al audio constituye una herramienta esencial en aplicaciones modernas de comunicación y entretenimiento. En este prototipo se desarrolla el diseño de un ecualizador digital implementado sobre un microcontrolador ESP32 [1] con soporte de FreeRTOS[2].

II. OBJETIVOS

Por consigna el objetivo es: diseñar un sistema de procesamiento digital de señales que permita filtrar una señal de audio en tiempo real dividiéndola en 3 bandas de frecuencia donde a cada una de esas bandas se le debe poder ajustar la ganancia en un rango de ± 10 dB y donde se debe poder ver su configuración actual. Además debe contar con: persistencia en la configuración; elementos de HMI; gestión remota de comandos; y un PCB prototipo.

III. DIAGRAMA EN BLOQUES

El siguiente diagrama en bloques muestra las conexiones que tendrá el prototipo entre módulos, bloques funcionales y el microcontrolador.

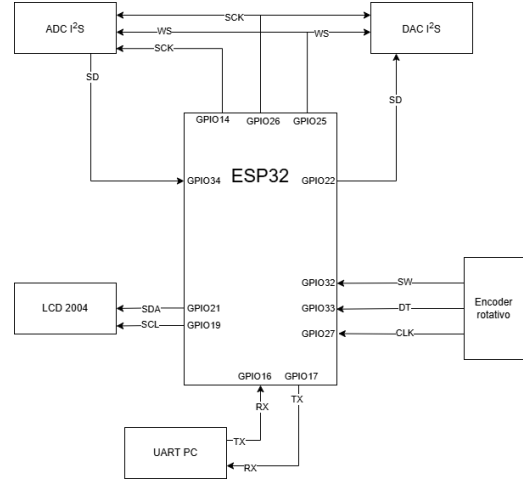


Fig. 1 diagrama en bloques

IV. HARDWARE

A. ADC I²S

El bloque funcional del ADC I²S estará construido en el PCB y está basado en el circuito integrado PCM1808 [3], es un ADC con protocolo de comunicación I²S con capacidad de funcionar en formato de 24 bits y frecuencias de muestreo que van de 8kHz a 96kHz y posibilidad de funcionar tanto como maestro o esclavo. En nuestro sistema funcionará como esclavo de la comunicación I²S, establecemos frecuencia de sampleo a 44.1kHz para muestrear la señal de entrada. Este se va a comunicar al microcontrolador mediante el hardware incluido I²S guardando los datos por DMA.



Fig. 2 integrado PCM1808

6.5 Electrical Characteristics

All specifications at $T_A = 25^\circ\text{C}$, $V_{CC} = 5\text{ V}$, $V_{DD} = 3.3\text{ V}$, master mode, $f_s = 48\text{ kHz}$, system clock = 512 f_s , 24-bit data, unless otherwise noted

PARAMETER	TEST CONDITIONS	MIN	TYP	MAX	UNIT
Resolution			24		Bits
DATA FORMAT					
Audio data interface format			I ² S, left-justified		
Audio data bit length			24		Bits
Audio data format			MSB-first, 2s complement		
f_s Sampling frequency		8	48	96	kHz
System clock frequency	256 f_s	2.048	12.288	24.576	MHz
	384 f_s	3.072	18.432	36.864	
	512 f_s	4.096	24.576	49.152	

Tabla 1 parámetros de funcionamiento PCM1808

B. DAC I²S

Para el DAC I²S se usará un módulo basado en el integrado PCM5102 [4], este integrado soporta resolución de palabra de 16, 24 y 32 bit, a fin de mantener la consistencia entre ADC y DAC usaremos el formato de 24 bit. También puede soportar frecuencias de muestreo que van de 8kHz a 384kHz, la cual nuevamente se configurará a 44.1kHz por consistencia entre dispositivos. Este al igual que el ADC funcionará como esclavo de la comunicación I²S y usará los datos procesados por DSP en el microcontrolador para generar a su salida una señal analógica. Según los parámetros que da su hoja de datos tiene como máximo una salida de 2.1 Vrms con una potencia máxima de salida de 6dBm (en condición de carga mínima de 1k Ω).

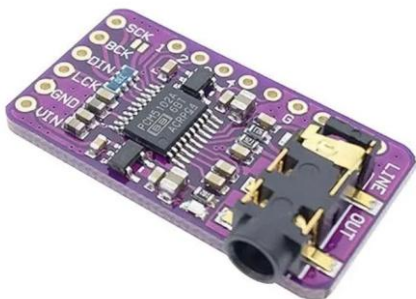


Fig. 3 módulo basado en PCM5102

PARAMETER	TEST CONDITIONS	MIN	TYP	MAX	UNIT
Resolution		16	24	32	Bits
Data Format (PCM Mode)					
Audio data interface format		I ² S, left justified			
Audio data bit length		16, 24, 32-bit acceptable			
Audio data format		MSB First, 2's Complement			
f _s ⁽¹⁾	Sampling frequency	8			
	System clock frequency	64, 128, 192, 256, 384, 512, 768, 1024, 1152, 1536, 2048, or 3072			
		f _{clock} up to 50Mhz			

Tabla 2 parámetros de funcionamiento PCM180

C. LCD 20x04

Como elemento visual de la HMI en el prototipo se usará un display LCD de 4 filas y 20 caracteres el cual indicará la configuración actual y también contará con un cursor que indica el parámetro que se quiere modificar actualmente. Este display cuenta con un módulo adaptador de comunicación por i²C basado en el integrado PCF8574T [5].



Fig. 4 frente y dorso del display LCD



Fig. 5 adaptador basado en PCF8574T

D. Encoder rotativo

Como elemento interactivo de la HMI en el prototipo se usará un encoder rotativo ky-040 [6], este al ser presionado cambia la posición del cursor que indica qué parámetro se quiere modificar mientras que girarlo hará que la ganancia de la banda seleccionada por el cursor suba o baje dependiendo de si se giró en sentido horario o antihorario respectivamente.

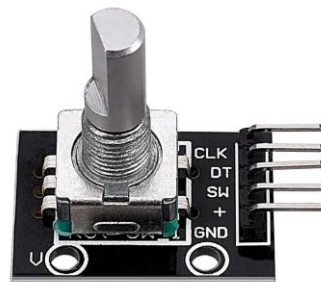


Fig. 6 encoder rotativo ky-040

E. UART PC

Para configuración remota de la ganancia de las bandas se podrá comunicar una PC por medio de protocolo UART al microcontrolador. Esta comunicación se hará por un puerto COM de la PC de forma cableada con un módulo conversor de USB a UART basado en el integrado PL2303 [7].

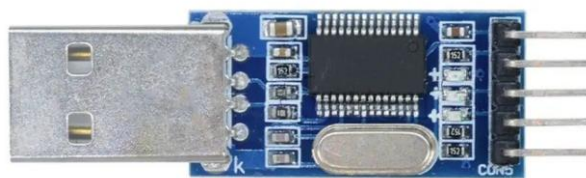


Fig. 7 adaptador basado en PL2303

F. ESP 32

Como centro de todo el diagrama estará el microcontrolador ESP32. Este se encargará de gestionar todos los periféricos del prototipo y del procesamiento de señales, todo por medio de un programa que hace uso de FreeRTOS



Fig. 8 placa desarrollo ESP32

VI. SOFTWARE

Desde el punto de vista del software el microcontrolador correrá un sistema operativo en tiempo real, más específicamente FreeRTOS provisto por ESPRESIF, el programa estará compuesto por un total de 6 tareas con 4 niveles de prioridad.

Primero determinaremos el funcionamiento del procesamiento y luego hablaremos propiamente de las tareas. Para hacer el procesamiento de señal el sistema consiste de un triple buffer rotativo, en simultáneo suceden 3 cosas: un buffer acumula las muestras entregadas por el ADC mediante el DMA del periférico I²S; un buffer es leído y procesado por algoritmos DSP con el núcleo del ESP32, para luego ser llenado con la información procesada; y el último buffer envía los datos procesados al DAC mediante el DMA del periférico. Finalmente cada vez que el ADC llena un buffer y a su vez el DAC vacía el suyo (por tener frecuencias y tamaño de palabra coherentes), los buffer rotan, pasando el lleno del ADC a ser procesado, el procesamiento del DSP a salir por el DAC y el vaciado del DAC a ser llenado por el ADC.

Audio Data Path

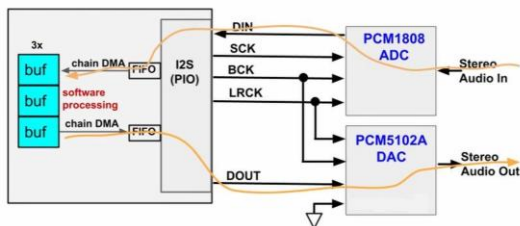


Fig. 9 datapath audio

Ahora si hablando de las tareas tenemos:

- **Task_I2S_Guardian:** esta tarea será la tarea de mayor prioridad (4) y la guardiana del periférico I²S (conectado a ADC y DAC) su rol principal por el funcionamiento del sistema será indicar al hardware I²S propio del ESP32 qué espacio de memoria debe escribir y leer por DMA, o sea, rotar los buffers. Y para con las otras tareas se encarga de entregarle a la tarea DSP un puntero de donde se encuentra el buffer de datos lleno del ADC. La sincronización entre la interrupción de llenado de buffer DMA y la tarea Guardián I²S se implementó mediante `xTaskNotifyFromISR`. Esta alternativa fue seleccionada

debido a que el evento posee un único consumidor y no requiere transferencia de datos asociada. Por sobre todo se eligió porque las Task Notifications presentan menor latencia [8] que un semáforo binario, resultando especialmente adecuada para nuestra aplicación de procesamiento de audio en tiempo real.

- **Task_DSP:** Esta tarea será la de segunda mayor prioridad (3), su finalidad será procesar los datos provistos del buffer del ADC por medio de filtros FIR propios de la librería `ESP_DSP` de `espressif`. Cada filtro tendrá sus coeficientes calculados por medio de una función dentro del programa que usa la fórmula provista en nota de aplicación de analog devices para un filtro `Windowed-Sinc` [9], la única diferencia es que la banda se genera como la resta de 2 filtros pasa bajos. Esta tarea a la salida de los filtros le aplicará la ganancia correspondiente a la configuración, la cual leerá con un `QueuePeek` de la cola de acceso común "param config queue" para finalmente ser re combinados en una sola. La sincronización de esta tarea será por un `xQueueReceive`, la cola proviene de la tarea guardiana del I²S y cuyo contenido es un puntero al buffer donde están los datos que envió el ADC.
- **Task_LCD:** Esta tarea estará en el tercer escalón de prioridad (2), su finalidad será mostrar en la pantalla LCD los parámetros actuales que serán leídos con un `xQueuePeek` "param config queue" del ecualizador así como también un cursor que indique cual es el parámetro a ser modificado. La sincronización de esta tarea será por un `vTaskDelayUntil` a la tasa de refresco de pantalla que establecemos en 1 segundo.
- **Task_encoder:** Esta tarea estará en el tercer escalón de prioridad (2), su finalidad será tomar los comandos dados por el encoder para modificar los parámetros actuales o mover el cursor que indica qué parámetro se está por modificar, estos se leerán/escribirán en la cola "param config queue" mediante `xQueuePeek` y `xQueueOverwrite` respectivamente. La sincronización de esta tarea será por un `xQueueReceiveFromISR` de una cola llamada "encoder queue" que tendrá información de qué hizo el encoder y en base a ello actuará. Esta tarea también entregará un semáforo llamado "change_semaphore" que indica que se hizo un cambio en la cola "param config queue".
- **Task_UART:** Esta tarea estará en el tercer escalón de prioridad (2), su finalidad será tomar los comandos dados por terminal de manera remota (PC) y en base a ellos hacer las modificaciones indicadas en "param config queue" mediante `xQueueOverwrite`. La sincronización de esta tarea será por medio de un `xQueueReceive` de una cola llamada "UART_queue" que contendrá la información enviada desde la terminal y en base a ello actuará. Esta tarea también entregará el semáforo llamado "change_semaphore" que indica que se hizo un cambio en la cola "param config queue".
- **Task_Flash:** Esta tarea estará en el último escalón de prioridad (1), su finalidad será tomar los parámetros actuales de "param config queue" y escribirlos en la memoria FLASH del microcontrolador para darle permanencia a la configuración entre apagados. La sincronización de esta tarea será por medio de un `xSemaphoreTake` de un semáforo llamado "change_semaphore", este lo entregan tareas que hayan modificado los parámetros de "param config queue".

VII. CONCLUSIONES

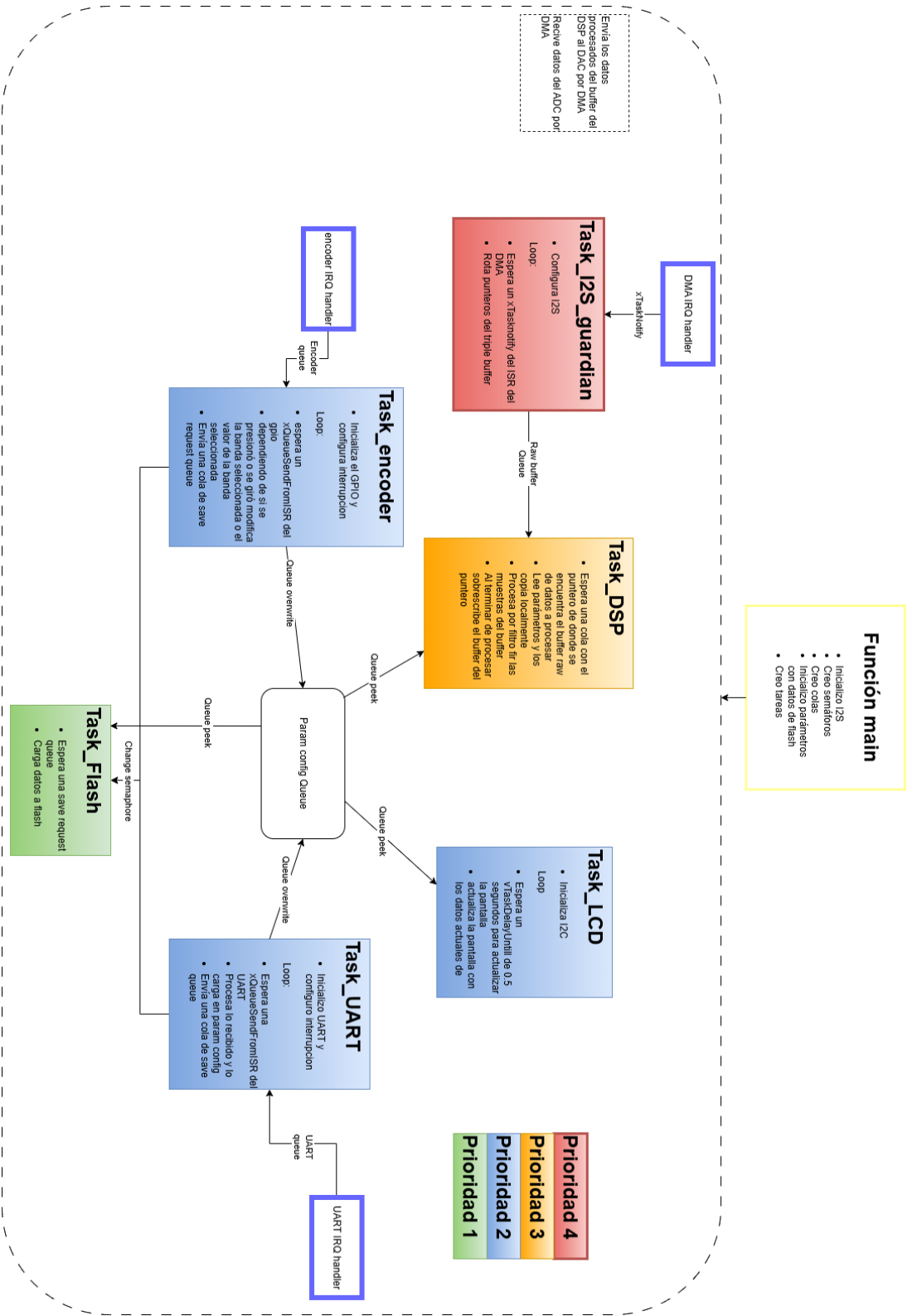
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Pellentesque ultrices pellentesque nulla id mollis. Vestibulum nec tempor ligula, in efficitur libero. Mauris malesuada ligula ac ullamcorper laoreet. Proin semper aliquam dictum. Etiam et diam eu est lobortis auctor a fermentum enim. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia curae; Sed et ullamcorper justo. Sed pharetra urna nisi, sed feugiat nunc malesuada vel. Proin vel tellus tempus, mattis metus non, mattis metus. Maecenas mi eros, congue ornare est volutpat, sollicitudin tincidunt metus. Quisque turpis ipsum, molestie in finibus vestibulum, gravida et massa. Curabitur placerat mattis nulla nec blandit.

Nunc vitae purus facilisis, egestas libero eget, imperdiet quam. Mauris consectetur, est ac faucibus auctor, purus odio semper sem, sit amet pulvinar orci metus pretium elit. Vestibulum pellentesque lacus tortor, lacinia suscipit nibh lobortis vel. Donec a accumsan ex. Aliquam placerat blandit erat, in maximus lorem mattis sed. Morbi sed augue eget metus rutrum egestas. Mauris a laoreet mi. Etiam congue lorem nec velit convallis, sed elementum augue mollis. Suspendisse suscipit risus vel sem feugiat, ac fringilla odio commodo. Phasellus non cursus enim, ac lacinia leo.

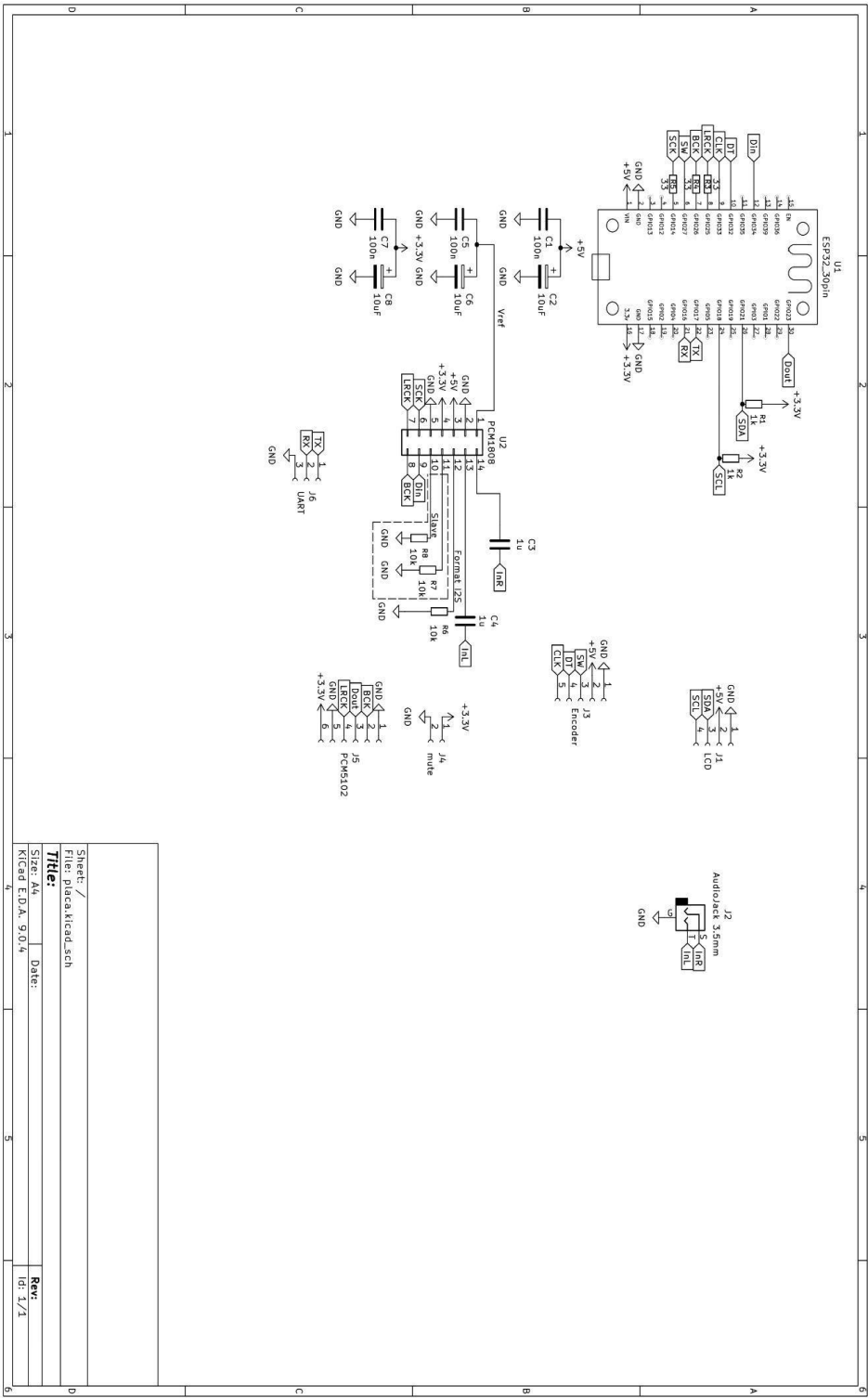
REFERENCIAS

- [1]. ESP 32
<https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32/esp32-devkitc/index.html>
- [2]. FreeRTOS (IDF)
https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/freertos_idf.html
- [3]. PCM1808
<https://www.alldatasheet.com/datasheet-pdf/view/2176993/TI2/PCM1808.html>
- [4]. PCM5102
<https://www.alldatasheet.com/datasheet-pdf/view/841405/TI1/PCM5102.html>
- [5]. PCF8574T
<https://www.alldatasheet.com/datasheet-pdf/view/18215/PHILIPS/PCF8574T.html>
- [6]. Encoder ky-040
<https://www.rcscomponents.kiev.ua/datasheets/ky-040-datasheet.pdf>
- [7]. PL2303
<https://www.alldatasheet.com/datasheet-pdf/view/1242559/PROLIFIC/PL2303.html>
- [8]. FreeRTOS direct to task notifications
<https://www.freertos.org/Documentation/02-Kernel/02-Kernel-features/03-Direct-to-task-notifications/01-Task-notifications>
- [9]. Windowed sinc coefficients
https://www.analog.com/media/en/technical-documentation/dsp-book/dsp_book_Ch16.pdf

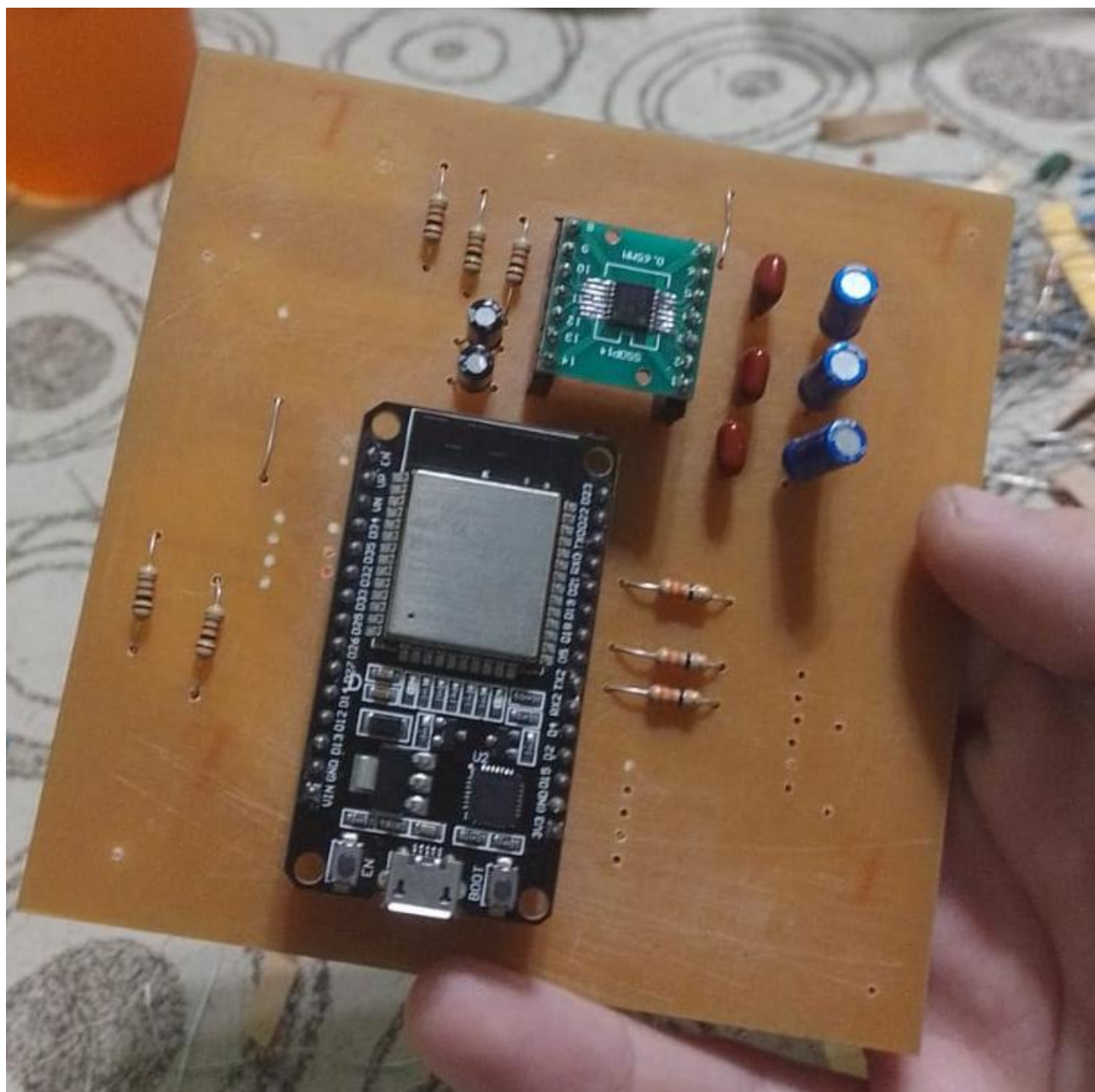
ANEXO 1 diagrama de tareas



ANEXO 2 esquemático



Anexo 4 PCB ensamblado



Anexo 5 diseño 3D carcasa dorso y frente

